

## Table of Contents

This Word document contains Heading 1-3 styles and a dynamic Table of Contents field. In Microsoft Word, right-click the table of contents and choose ?Update Field? before final submission to refresh page numbers after any edits.

Report structure: Acknowledgements; eProject Synopsis; eProject Analysis; eProject Design; diagrams; proposed database design; screenshot plan; source-code commentary; user guide; developer guide; modules; testing; conclusion.

## Acknowledgements

I acknowledge the guidance of my lecturers, supervisors, and peers whose teaching and feedback supported the completion of this eProject. I also acknowledge the tools and resources used in developing the Fanimation interface. This report has been prepared from the delivered React source code and is intended to present the work accurately and professionally.

## 1. eProject Synopsis

### 1.1 Introduction

Fanimation is a responsive single-page web application that presents a catalogue of fan products and related accessories. It combines a marketing-style landing page with interactive product discovery, product-detail overlays, favourites, a client-side cart, a gallery, offers, frequently asked questions, and a validated contact form.

### 1.2 Background

The implementation models a consumer-facing fan catalogue. Product records are held locally in JavaScript and rendered in React. The interface communicates pricing in Nigerian naira (N) and includes catalogue categories for ceiling, pedestal, wall and exhaust fans, plus accessories.

### 1.3 Purpose and Objectives

Present a visually organised fan product catalogue.

Allow users to filter and sort the available products.

Allow users to view richer product details, mark favourites, and manage a temporary cart.

Provide supporting promotional, gallery, company, FAQ and contact sections in one scrollable experience.

Demonstrate a responsive React component architecture using Vite, Bootstrap and custom CSS.

### 1.4 Scope

The delivered application is front-end only. It renders 15 local product records and images. It does not include authentication, a backend API, database, search behaviour, payment execution, checkout processing, order persistence, or message delivery. The visible Checkout button has no click handler; the contact form validates input and shows a local success state only.

### 1.5 Technologies Used

### 1.6 Expected Outcome

A visitor can browse a responsive Fanimation storefront, narrow its local product catalogue, inspect products in a modal, manage a temporary cart and use the contact-form interaction. These outcomes are implemented as client-side interactions only.

## 2. eProject Analysis

## 2.1 Existing System

No separate legacy system is present in the uploaded repository. The delivered application itself is the system available for analysis. It behaves as a static, front-end catalogue and promotional site.

## 2.2 Problems Addressed

The interface addresses the practical discovery problem of presenting multiple fan types in a single catalogue. Category, type, colour, brand, price ceiling and sort controls reduce the effort required to browse the 15 available records.

## 2.3 Proposed System

The proposed and implemented system is a React single-page application. It centralises catalogue information in `src/data/products.js`, composes the page from reusable modules, and keeps transient user interactions in React state.

## 2.4 Functional Requirements - Verified

## 2.5 Non-functional Requirements - Observed

Responsive layout through Bootstrap breakpoints and local media queries.

Smooth anchor scrolling with `scroll-margin` for section targets.

Basic accessibility measures: semantic sections, labels, alt text and aria-labels on important controls.

Fast local data access because the catalogue and assets are bundled with the client.

Maintainability through independently styled component folders.

## 2.6 Assumptions and Constraints

Catalogue prices are comma-formatted strings and are converted for comparison and calculation. Delivery, warranty, showroom and company statements are presentation content, not integrated business services. `LocalStorage` stores a browser-local visitor counter; `SessionStorage` prevents incrementing it more than once in the current browser session. No persistence is provided for favourites or cart data.

## 2.7 Feasibility Analysis

# 3. eProject Design

## 3.1 Overall System Architecture

The browser loads `index.html`, which mounts `main.jsx` into `#root`. `main.jsx` imports Bootstrap and global CSS, then renders `App`. `App` composes the site sections and owns cross-section favourite, cart and cart-drawer state. `ProductSection` owns the filter state and derives filtered results from `allProducts`.

### Figure 1. Component and data architecture

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
graph TD
    Browser --> index[index.html]
    index --> main[src/main.jsx]
    main --> App
    App --> Navbar
    App --> Ticker
    App --> Hero
    App --> Features
    App --> ProductSection
    App --> CartDrawer
    App --> Gallery
    App --> Offers
    App --> About
    App --> FAQ
    App --> Contact
    App --> Footer
    ProductSection --> ProductFilter
    ProductSection --> ProductGrid
    ProductGrid --> ProductCard
    ProductGrid --> ProductModal
    Products["products.js: local array"] --> ProductSection
    Products --> Gallery
    Products --> Offers
    AppState["AppState: (favourites/cart state)"] --> Navbar
    AppState --> ProductSection
    AppState --> CartDrawer
```

## 3.2 Folder Structure

|                                                               |                                                     |
|---------------------------------------------------------------|-----------------------------------------------------|
| <code>src/ assets/images/</code>                              | hero, logo and 15 product photos                    |
| <code>components/</code>                                      | 14 feature/component folders                        |
| <code>ProductSection/ProductGrid/ProductCard</code>           | <code>data/products.js</code>                       |
| catalogue, <code>allProducts</code> and <code>toNumber</code> | <code>styles/global.css</code>                      |
| <code>scroll/focus styling</code>                             | <code>App.jsx</code>                                |
| and shared state                                              | <code>main.jsx</code>                               |
| Bootstrap imports                                             | <code>public/</code>                                |
|                                                               | global application composition                      |
|                                                               | React bootstrap and                                 |
|                                                               | <code>favicon.svg</code> and <code>icons.svg</code> |

package.json                      scripts and dependencies vite.config.js

React plugin configuration

### 3.3 Navigation and Application Flow

Navbar and footer links use in-page anchors. Product-category links dispatch a CustomEvent named fanimation:filter-category; ProductSection listens for it, updates category filter state and the page scrolls to the products section. The Hero button scrolls to products directly. On mobile, Navbar displays an overlay panel controlled by isMenuOpen.

### 3.4 State Management

### 3.5 Data Flow and Filtering

allProducts is the authoritative current catalogue. ProductSection derives the maximum price and option lists with useMemo. It filters by exact category/type/colour/brand match and numeric price ceiling. It then sorts a copied result set. Changing category clears a previously selected type because that type may no longer be valid.

### 3.6 Styling and Responsive Design

Bootstrap supplies the container/row/column grid and responsive breakpoints. Each module imports its own CSS file. Gallery changes from four columns to three below 992px and two below 576px. Navbar switches to a mobile overlay at 991px and below. About stats stack at 576px. Modal padding/image size reduce at 576px. Product columns use col-lg-4 col-md-6, producing three columns on large screens and two on medium screens before stacking on small screens.

## 4. Data Flow Diagrams, Flowcharts and Process Diagrams

### 4.1 Context Diagram

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart LR
    U[Visitor] -->|browse, filter, favourite, add to cart, submit contact form| S[Fanimation SPA]
    S -->|catalogue, UI feedback, local cart/favourite state| U
    S -->|read/write visitor count| LS[(Browser localStorage/sessionStorage)]
    LS --> D[Bundled product data/assets]
    D --> S
    S -- note[No API, database, email or payment gateway is connected] -. -> S
```

### 4.2 Level 1 DFD

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart TD
    U[Visitor] --> N[Navigation process]
    N --> P[Catalogue/filter process]
    P --> C[Cart/favourite process]
    C --> F[Contact validation process]
    F --> V[Rendered sections]
    V --> P
    P <--> D[(products.js)]
    D --> V
    V --> C
    C --> M[(React App state)]
    M --> V
    V --> F
    F --> V
    V <--> L[(localStorage/sessionStorage visitor counter)]
```

### 4.3 Level 2 DFD - Catalogue filtering

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart TD
    I[Filter input or category event] --> S[Update filters state]
    S --> T{Category changed?}
    T -->|yes| R[Reset type]
    T -->|no| X[Keep type]
    R --> F[Filter allProducts]
    X --> F
    F --> G[Category/type/colour/brand/price tests]
    G --> O[Sort selected result]
    O --> K[Render ProductGrid or no-results message]
```

### 4.4 Flowchart - Application startup

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart TD
    A[Open site] --> B[index.html loads]
    B --> C[main.jsx imports Bootstrap/global CSS]
    C --> D[createRoot renders App in StrictMode]
    D --> E[Sections and local catalogue render]
    E --> F[Navbar visitor effect and ticker timer begin]
```

### 4.5 Flowchart - Homepage navigation

Mermaid source - render in a Mermaid-capable editor if a graphical figure is

required by the centre.

```
flowchart TD
  A[Click nav/footer link] --> B{Category link?}
  B -->|yes| C[Dispatch fanimation:filter-category]
  B -->|no| D[Find target section]
  C --> D
  D --> E[Smooth scroll]
  E --> F[Mobile panel closes when used]
```

#### 4.6 Flowchart - Product browsing and filtering

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart TD
  A[Open products] --> B[Show all 15 records sorted by rating]
  B --> C[Choose filter or sort]
  C --> D[Update filters]
  D --> E[Derive matching records]
  E --> F{Any match?}
  F -->|yes| G[Render cards]
  F -->|no| H[Show adjustment message]
```

#### 4.7 Flowchart - Product details modal

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart TD
  A[Click View Details] --> B[showModal=true]
  B --> C[Render modal and lock body scroll]
  C --> D[Action]
  D -->|Add to Cart| E[App increments/adds item and opens drawer]
  D -->|Favourite| F[Toggle product ID]
  D -->|Close/backdrop/Escape| G[Unmount modal and restore scroll]
```

#### 4.8 Flowchart - Contact process

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart TD
  A[Enter form fields] --> B[Submit]
  B --> C[Prevent default]
  C --> D[Validate name, email and message]
  D --> E{Errors?}
  E -->|yes| F[Display inline errors]
  E -->|no| G[Show success message and clear form]
  G --> H[No transmission occurs in current implementation]
```

#### 4.9 Overall user flow

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart LR
  Start --> Home
  Home --> Products
  Products --> Details
  Details --> Cart
  Cart --> Home
  Home --> Gallery
  Home --> Offers
  Offers --> Products
  Products --> FAQ
  FAQ --> Contact
  Contact --> Cart
  Cart --> End[Checkout button visible; no checkout workflow implemented]
```

#### 4.10 Process - Product viewing

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart LR
  Catalogue[products.js] --> Grid[ProductGrid]
  Grid --> Card[ProductCard]
  Card --> Modal[ProductModal]
  Modal --> Info[Image, rating, description, specs, price and perks]
```

#### 4.11 Process - Search/filtering

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart LR
  Note[No text-search input/action is implemented] --> Filters[Category/type/price/colour/brand controls]
  Filters --> Results[Filtered and sorted cards]
```

#### 4.12 Process - Modal, navigation, contact and complete workflow

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
flowchart TD
  Navigation --> Catalogue
  Catalogue --> Modal
  Modal --> Cart
  Navigation --> Gallery
  Gallery --> Lightbox
  Navigation --> Contact
  Contact --> Validation
  Validation --> Confirmation
  Cart --> LocalOnly[Temporary browser state only]
```

## 5. Database Design / Structure

### 5.1 Current versus Proposed Design

Current implementation: no database exists. The catalogue is a static JavaScript array; reviews, ratings and prices are display fields in that array. Browser storage is used only for the visitor counter and session marker. The following is a proposed normalized production design, clearly separate from the delivered implementation.

### 5.2 Proposed production ER Diagram

Mermaid source - render in a Mermaid-capable editor if a graphical figure is required by the centre.

```
erDiagram
    CATEGORY ||--o{ PRODUCT : classifies
    PRODUCT ||--o{ REVIEW : receives
    USER ||--o{ REVIEW : writes
    USER ||--o{ WISHLIST : owns
    PRODUCT ||--o{ WISHLIST : saved
    USER ||--o{ ORDER : places
    ORDER ||--|{ ORDER_ITEM : contains
    PRODUCT ||--o{ ORDER_ITEM : ordered
    USER ||--o{ CONTACT_MESSAGE : sends
    CATEGORY {
        int category_id PK
        string name UK
    }
    PRODUCT {
        int product_id PK
        int category_id FK
        string name
        decimal price
        decimal old_price
        decimal rating
        int review_count
    }
    USER {
        int user_id PK
        string email UK
        string full_name
    }
    REVIEW {
        int review_id PK
        int product_id FK
        int user_id FK
        tinyint rating
        text comment
    }
    WISHLIST {
        int wishlist_id PK
        int user_id FK
        int product_id FK
    }
    ORDER {
        int order_id PK
        int user_id FK
        decimal subtotal
        string status
    }
    ORDER_ITEM {
        int order_item_id PK
        int order_id FK
        int product_id FK
        int quantity
        decimal unit_price
    }
    CONTACT_MESSAGE {
        int message_id PK
        int user_id FK
        string name
        string email
        text message
    }
```

## 5.3 Proposed Table Structures and Constraints

### 5.4 Normalization

The proposed schema is in third normal form: categories are separated from products; user identity is separated from reviews, wishlist rows and orders; and order lines hold the repeatable product/quantity relationship. Prices in order\_items intentionally preserve transactional history rather than relying on a future products.price value.

## 6. Screenshot Placeholders

The following figure placeholders identify the recommended screenshots for a final submission. Capture them from the running application at the stated viewport before inserting. The source repository contains no submitted screenshots.

Figure 1. Homepage and Hero Banner. Top bar, navigation, hero message and Explore Collection call to action.

Figure 2. Navigation and Mobile Menu. Desktop navigation plus the overlay panel at a narrow viewport.

Figure 3. Product Catalogue. Filter sidebar, product cards, product count and benefit panel.

Figure 4. Product Filters. Category, type, price slider, colour chips, brand and clear action.

Figure 5. Product Details Modal. Product image, specifications, rating, add-to-cart and wishlist actions.

Figure 6. Cart Drawer. Temporary cart line item, quantity controls and subtotal.

Figure 7. Gallery. Eight-image responsive gallery and selected-image lightbox.

Figure 8. Offers. Products having oldPrice with calculated percentage discount.

Figure 9. About and FAQ. Company text/stat cards and Bootstrap accordion.

Figure 10. Contact and Footer. Contact details, validated form, confirmation state and footer links.

Figure 11. Tablet View. Bootstrap medium-width arrangement with two product columns.

Figure 12. Mobile View. Small-screen stacked product layout and mobile navigation.

## 7. Source Code with Comments

### 7.1 Commenting Approach

The reviewed source already includes focused comments around non-obvious behaviour, notably price conversion, category-event communication, modal lifecycle handling, gallery asset reuse and contact-form limitations. The code is not over-commented. The snippets below explain the important logic without rewriting the full source.

### 7.2 Shared Cart State - App.jsx

```
const handleAddToCart = (product) => {
  const normalizedProduct = {
```

```
...product, name: product?.name || product?.title || "Product", price:
toNumber(product?.price ?? product?.cost ?? product?.amount ?? 0), );
setCartItems((prev) => { const existing = prev.find((item) => item.id ===
normalizedProduct.id); if (existing) return prev.map((item) =>
item.id === normalizedProduct.id ? { ...item, quantity: item.quantity
+ 1 } : item); return [...prev, { ...normalizedProduct, quantity: 1 }];
}); setIsCartOpen(true); });
```

This function normalizes flexible product-like inputs, increments an existing cart line or appends a new one, then opens the drawer. It is held in App so NavBar, ProductSection and CartDrawer can share the same state.

### 7.3 Derived Filtering - ProductSection.jsx

```
let result = allProducts.filter((product) => { if (filters.category &&
product.category !== filters.category) return false; if (filters.type &&
product.type !== filters.type) return false; if (filters.color &&
product.color !== filters.color) return false; if (filters.brand &&
product.brand !== filters.brand) return false; if (filters.price &&
toNumber(product.price) > Number(filters.price)) return false; return true;
}); // result is then copied and sorted by rating, id or numeric price.
```

The predicate only applies a criterion when its filter value is present.

toNumber removes commas from stored price strings before numeric comparison.

### 7.4 Modal Lifecycle - ProductModal.jsx

```
useEffect() => { const handleKeyDown = (e) => { if (e.key === "Escape")
onClose(); }; document.addEventListener("keydown", handleKeyDown);
document.body.style.overflow = "hidden"; return () => {
document.removeEventListener("keydown", handleKeyDown);
document.body.style.overflow = ""; }; }, [onClose]);
```

When mounted, the modal listens for Escape and disables page scrolling. Its cleanup restores both; backdrop clicks call onClose while clicks within the modal stop propagation.

### 7.5 Contact Validation - Contact.jsx

```
const validate = (values) => { const nextErrors = {}; if
(!values.name.trim()) nextErrors.name = "Please enter your name."; if
(!values.email.trim()) nextErrors.email = "Please enter your email."; else
if (!/^S+@\S+\.\S+$/i.test(values.email)) nextErrors.email = "Please enter
a valid email address."; if (!values.message.trim()) nextErrors.message =
"Please enter a message."; return nextErrors; };
```

The validation runs entirely in the browser. A valid submission clears the fields and displays a confirmation but does not send or store a message.

## 8. User Guide

### 8.1 Introduction and System Requirements

Fanimation is used through a modern web browser with JavaScript enabled. No user account is required for the current demonstrator. Internet access is only needed when the app is deployed or served remotely; the submitted source can run locally with Node.js and npm.

### 8.2 Starting the Application

A non-technical user normally opens the deployed site in a browser. For a local demonstration, a developer starts the Vite development server; then the user opens the displayed local address.

### 8.3 Using the Homepage and Navigation

Use the navigation bar or footer links to jump to a section. Choose a product-category link such as CEILING FANS to move to the catalogue with that category already selected. On a narrow screen, use the hamburger button to open the navigation panel. The Explore Collection button moves to products.

### 8.4 Browsing, Filtering and Sorting Products

At Products, use Category, Type, Price Range, Color and Brand controls. Colour circles toggle a colour selection. Choose a Sort By option to change the

order. Use Clear Filters to return to all categories, all types/colours/brands, the maximum price and Popular sort. If nothing matches, the page says to adjust or clear filters.

## 8.5 Product Details, Favourites and Cart

Select View Details to open a product modal. Read the specifications and price, then choose Add to Cart or the heart button. Close using the X, by clicking outside the panel, or pressing Escape. The cart icon opens the cart drawer. Use minus/plus buttons to change quantities; reducing a line from one to zero removes it. Checkout is presently a visual button only and does not complete a purchase.

## 8.6 Gallery, Offers, FAQs and Contact

Select a gallery image to enlarge it; click the overlay to close. Offers show the items whose local data has an old price. Expand FAQ questions to read their answers. In Contact, enter a name, a syntactically valid email and a message; submit to receive a local confirmation. The message is not sent to an organisation in the current version.

## 8.7 Troubleshooting and Closing

# 9. Developer Guide

## 9.1 Installation and Commands

npm install npm run dev npm run build npm run preview npm run lint  
package.json defines these scripts. The project uses ESM ("type": "module").  
Vite configuration registers only the React plugin. Bootstrap CSS and bundle JS are imported at startup so the FAQ collapse behaviour is available.

## 9.2 Component Hierarchy and Dependencies

App is the composition root. ProductSection depends on ProductFilter and ProductGrid; ProductGrid depends on ProductCard; ProductCard conditionally renders ProductModal. App also imports CartDrawer. Components import their paired CSS and Lucide icons where required. allProducts is read by ProductSection, Gallery and Offers.

## 9.3 Adding or Editing a Product

Edit src/data/products.js. Add an object with a unique numeric id, name, category, type, color, brand, comma-formatted price, rating, reviews, badge (if needed), imported image and description. oldPrice is optional; when present it causes Offers to include that record. Add a corresponding colour mapping in ProductSection.jsx if a new colour should show as an accurate filter swatch.

## 9.4 Extending Features

For persistence, replace local arrays with an API-backed data layer and move cart/favourites to authenticated server state or local storage. For functional contact, submit to a secure endpoint and surface asynchronous failure/success. For checkout, add a verified payment flow, server-side price calculation and order creation. Text search is not implemented and should be designed as a separate filter criterion rather than describing the present filter UI as search.

## 9.5 Styling, Debugging and Maintenance

Keep reusable styles within the component folder and reserve global.css for truly global rules. Preserve Bootstrap breakpoint classes where the layout depends on them. Verify modal cleanup, category-event listener cleanup and timer cleanup after changes. Use the Vite console and browser developer tools for runtime issues. Keep product price display and numeric conversion compatible, or migrate prices to numeric values at the API boundary.

## 9.6 Build and Deployment

The intended build command is npm run build, which invokes vite build and normally emits a deployable bundle to dist. During this report's verification, the installed dependency set produced a Vite/Rolldown build error concerning

an absolute/relative emitted index.html name. Resolve that environment/dependency issue and rerun the build before deploying; do not treat the existing dist directory as proof that the present build passes.

## **10. Module Descriptions**

## **11. Testing**

### **11.1 Verification Performed**

The source was inspected module-by-module and the configured build and lint commands were executed in the delivered workspace. No automated test suite is declared in package.json and no test files were found. Therefore, the functional/UI cases below are documented as manual test cases derived from code review and intended browser verification; they are not represented as an invented automated test run.

### **11.2 UI, Responsive, Cross-browser and Performance Testing**

The project includes responsive rules but no committed device/browser test evidence or performance benchmark. Recommended completion testing is Chrome, Firefox, Edge and Safari at desktop, tablet and mobile widths; validate navigation, FAQ collapse, modal, lightbox, cart and contact states. Production performance should be measured only after the build failure has been resolved.

### **11.3 Bugs and Fixes / Observations**

Observed build issue: vite build failed with a Rolldown error that an emitted index.html name was treated as an absolute/relative path. Observed lint issue: Navbar sets visitorCount synchronously in an effect, which the configured React Hooks lint rule rejects. These are report findings, not unverified claims that fixes have been applied. A functional limitation is that social anchors use # and do not target external social destinations.

## **12. Conclusion and Future Improvements**

### **12.1 Objectives Achieved**

The delivered project successfully demonstrates a coherent responsive React catalogue experience with 15 products, multi-criterion filtering, sorting, product-modal details, favourites, a temporary cart, gallery, offers, FAQ and contact validation. Its folder structure and local data model make the interface understandable and extendable.

### **12.2 Challenges and Lessons Learned**

The project highlights the distinction between a convincing front-end interaction and a production transaction system. Local component state is effective for demonstration but is not durable. Storing formatted prices as strings also requires deliberate conversion before filtering and calculations. Cross-component interactions are managed by lifted state and a small custom browser event, avoiding a global state library.

### **12.3 Recommendations and Future Enhancements**

Resolve the current build/lint findings and add automated component and end-to-end tests.

Add a backend API and normalized database based on the proposed schema. Persist users, favourites, carts, contact messages and orders securely.

Implement authenticated checkout, payment integration, server-side price validation and order history.

Implement an actual product text search if required; it is not present now.

Add accessible keyboard/focus management for custom overlays, and conduct formal cross-browser/performance testing.

Replace placeholder company/contact information and capture final screenshots after deployment.

### **12.4 Documentation Standards**

This report distinguishes verified implementation from recommended production work. It does not claim backend services, persistence, payment, search or



message delivery where the submitted source does not implement them. Before submission, the student should replace cover-page placeholders, update the automatic Table of Contents in Word, insert the numbered running-application screenshots and rerun build/lint after correcting the reported issues.

Submission information | Value

Student Name | [Insert student name]

Student ID | [Insert student ID]

Centre | [Insert centre]

Course | [Insert course]

Semester | [Insert semester]

Session | [Insert academic session]

Submission Date | [Insert submission date]

Technology | Verified use

React 19 | Component rendering and local state.

Vite 8 | Development server and configured production build tool.

Bootstrap 5 | Grid, form, button, accordion and responsive utility classes.

Lucide React | Interface icons.

CSS | Component-scoped stylesheet files plus global smooth-scrolling/focus overrides.

JavaScript modules | Catalogue data, components and helper functions.

ID | Requirement | Implementation status / evidence

FR-01 | Display a promotional homepage | Implemented: Navbar, Ticker, Hero and Features are rendered by App.

FR-02 | Browse products | Implemented: ProductGrid maps the filtered local catalogue to cards.

FR-03 | Filter products | Implemented: category, dependent type, price, colour and brand controls.

FR-04 | Sort products | Implemented: popular (rating), newest (descending id), price ascending and descending.

FR-05 | View details | Implemented: ProductCard opens ProductModal.

FR-06 | Manage favourites | Implemented in memory via favourite product IDs and visible count.

FR-07 | Manage cart | Implemented in memory: add, quantity +/- and subtotal drawer.

FR-08 | View promotions and gallery | Implemented: oldPrice-derived offers and eight-image lightbox gallery.

FR-09 | Read FAQs | Implemented: Bootstrap collapse accordion.

FR-10 | Contact interaction | Implemented: client-side required/email validation and confirmation state.

Area | Assessment

Technical | Feasible for a demonstrator: the client-only architecture has few infrastructure dependencies.

Operational | Usable for browsing and enquiry demonstration; not operationally complete for sales because checkout and contact delivery are absent.

Economic | Low running complexity for a static deployment; a production service would require API, database, security and payment costs.

Schedule | The component/data separation permits focused incremental enhancement.

State owner | State | Purpose

App | favoriteIds | Array of product IDs; toggled by cards/modal and shown in Navbar.

App | cartItems, isCartOpen | Cart records/quantities and drawer visibility.

ProductSection | filters | Category, type, price ceiling, colour, brand and sort selection.

ProductCard | showModal | Whether its detail modal is rendered.

Gallery | activeImage | Selected lightbox image.

Contact | form, errors, submitted | Controlled form, validation feedback and local success message.

Navbar | visitorCount, isMenuOpen | Browser-local counter and mobile menu visibility.

Ticker | now | Live one-second clock.

Table | Key fields and types | Constraints/relationships

categories | category\_id INT PK; name VARCHAR(80) | name UNIQUE, NOT NULL.

products | product\_id INT PK; category\_id INT FK; name VARCHAR(180); type VARCHAR(120); colour VARCHAR(60); brand VARCHAR(80); price DECIMAL(12,2); old\_price DECIMAL(12,2) NULL; rating DECIMAL(2,1); description TEXT | price >= 0; old\_price > price when supplied; belongs to one category.

users | user\_id INT PK; full\_name VARCHAR(120); email VARCHAR(254) | email UNIQUE; required for account-based features.

reviews | review\_id INT PK; product\_id INT FK; user\_id INT FK; rating TINYINT;

comment TEXT | rating between 1 and 5; one product and one author.

contact\_messages | message\_id INT PK; user\_id INT FK NULL; name VARCHAR(120); email VARCHAR(254); message TEXT; created\_at TIMESTAMP | name/email/message required; optional linked user.

wishlists | wishlist\_id INT PK; user\_id INT FK; product\_id INT FK; created\_at TIMESTAMP | UNIQUE(user\_id, product\_id).

orders | order\_id INT PK; user\_id INT FK; subtotal DECIMAL(12,2); status VARCHAR(30); created\_at TIMESTAMP | subtotal >= 0; belongs to one user.

order\_items | order\_item\_id INT PK; order\_id INT FK; product\_id INT FK; quantity INT; unit\_price DECIMAL(12,2) | quantity > 0; preserves price at time of order.

[ SCREENSHOT PLACEHOLDER ] Homepage and Hero Banner

[ SCREENSHOT PLACEHOLDER ] Navigation and Mobile Menu

[ SCREENSHOT PLACEHOLDER ] Product Catalogue

[ SCREENSHOT PLACEHOLDER ] Product Filters

[ SCREENSHOT PLACEHOLDER ] Product Details Modal

[ SCREENSHOT PLACEHOLDER ] Cart Drawer

[ SCREENSHOT PLACEHOLDER ] Gallery

[ SCREENSHOT PLACEHOLDER ] Offers

[ SCREENSHOT PLACEHOLDER ] About and FAQ

[ SCREENSHOT PLACEHOLDER ] Contact and Footer

[ SCREENSHOT PLACEHOLDER ] Tablet View

[ SCREENSHOT PLACEHOLDER ] Mobile View

Issue | Action

Product list appears empty | Use Clear Filters.

Modal will not close | Press Escape, click the close X, or click outside the panel.

Cart contents disappear | This is expected after a page refresh because cart state is not persisted.

Contact confirmation expected by email | The app does not transmit email; the confirmation is visual only.

Close the site | Close the browser tab. For local hosting, stop the Vite terminal with Ctrl+C.

Module | Purpose, inputs/outputs and key relationships

main.jsx | Imports Bootstrap/global CSS and mounts App in React StrictMode.

App.jsx | Top-level composition; owns favourites, cart and cart-drawer state; passes callbacks/counts.

Navbar | Anchor navigation, category CustomEvent dispatch, visitor counter, mobile overlay, favourite/cart indicators.

Ticker | Scrolling announcements and one-second display clock.

Hero | Hero asset, marketing content and scroll-to-products action.

Features | Four static feature cards.

ProductSection | Filter state, option derivation, CustomEvent listener, filtering/sorting and catalogue layout.

ProductFilter | Controlled filter controls; returns changes to ProductSection.

ProductGrid | Maps matching records to cards or shows no-results state.

ProductCard | Renders a product summary, favourite action and per-card modal state.

ProductModal | Displays detail; manages Escape/body-scroll lifecycle; triggers cart/favourite callbacks.

CartDrawer | Normalizes cart rows, calculates subtotal and exposes quantity/close actions.

Gallery | Derives first eight product images and controls a lightbox.

Offers | Selects oldPrice records and calculates percentage reductions.

About | Static company narrative and four statistics.

FAQ | Static FAQ data in Bootstrap accordion markup.

Contact | Controlled validation form and local confirmation state.

Footer | Static links, category filter dispatch and contact/social presentation.

products.js | 15 local records; named allProducts and toNumber exports; default first-three subset.

CSS files | Component styling, motion and local responsive overrides.

Test ID | Test case | Expected result | Actual / status

T-01 | Load page | All sections render in page order. | Code-reviewed; implementation present.

T-02 | Choose a category navigation link | Products section scrolls and category filter updates. | Code-reviewed; CustomEvent/listener present.

T-03 | Apply filters/sort | Only matching product cards display in requested order. | Code-reviewed; predicate/sort logic present.

T-04 | Open/close details | Modal opens; X/backdrop/Escape close it and restore scroll. | Code-reviewed; lifecycle present.

T-05 | Add/adjust cart item | Item is added/incremented; zero quantity removes it; subtotal updates. | Code-reviewed; state logic present.

T-06 | Submit invalid contact form | Inline errors appear. | Code-reviewed; validate() present.

T-07 | Submit valid contact form | Confirmation displays and fields clear; no message transmission. | Code-reviewed; local behaviour present.

T-08 | Responsive UI | Grid/menu/modal changes at defined breakpoints. | Static CSS/Bootstrap review complete; browser matrix not executed.

T-09 | npm run build | Production bundle should build. | Failed in inspected environment: Vite/Rolldown emitted-name error.

T-10 | npm run lint | No lint errors expected. | Failed: react-hooks/set-state-in-effect in Navbar.jsx line 40.